CSci 135: Module 1 →

Software Evolution

*

Version: v4.2026

Rodney Maniego Jr.
Department of Computer Science and Technology
Faculty of Computing * Visayas State University



Disclaimer: This presentation is specially prepared as part of the **CSci 135 Software Engineering I** course for the **Visayas State University**. It is provided for educational purposes under fair use. Unauthorized copying, distribution, or reproduction of this material, in whole or in part, is strictly prohibited.

*

Course Outcome

Explain the concepts, principles, and professional context of software engineering

Learning Outcomes

Discuss the evolution, characteristics, categories, and professional roles of software.

Outline

1. Business Context
2. Course Overview
3. Software Evolution
4. Software Characteristics
5. Software Classifications
6. Roles in SW Development

Software Evolution

Business Context

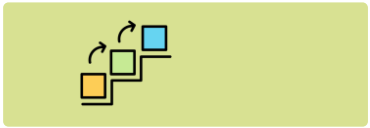
Business Context

2026 and onwards



AI-Native Business

Business as usual, but autonomous agents become embedded in products and workflows.



Evolving SE Roles

AI agents amplify the strengths of high-performing organizations and worsen the flaws of immature teams.



Responsible SW

Security, reliability, governance, and verification become increasingly integral in agentic code bases.



Side Effects

AI economics impact localized electricity costs and global computer resource prices.

https://dora.dev/insights/balancing-ai-tensions/
https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/tech-forward/state-of-ai-trust-in-2026-shifting-to-the-agentic-era

Business Context

2026 and onwards



Existing Impacts

Software still impacts human life and infrastructure.



Software Crisis

Inexperienced "programmers" repeat software crisis.



Faster Delivery

Deliver high-quality software that meets customer needs at a faster rate without compromise.

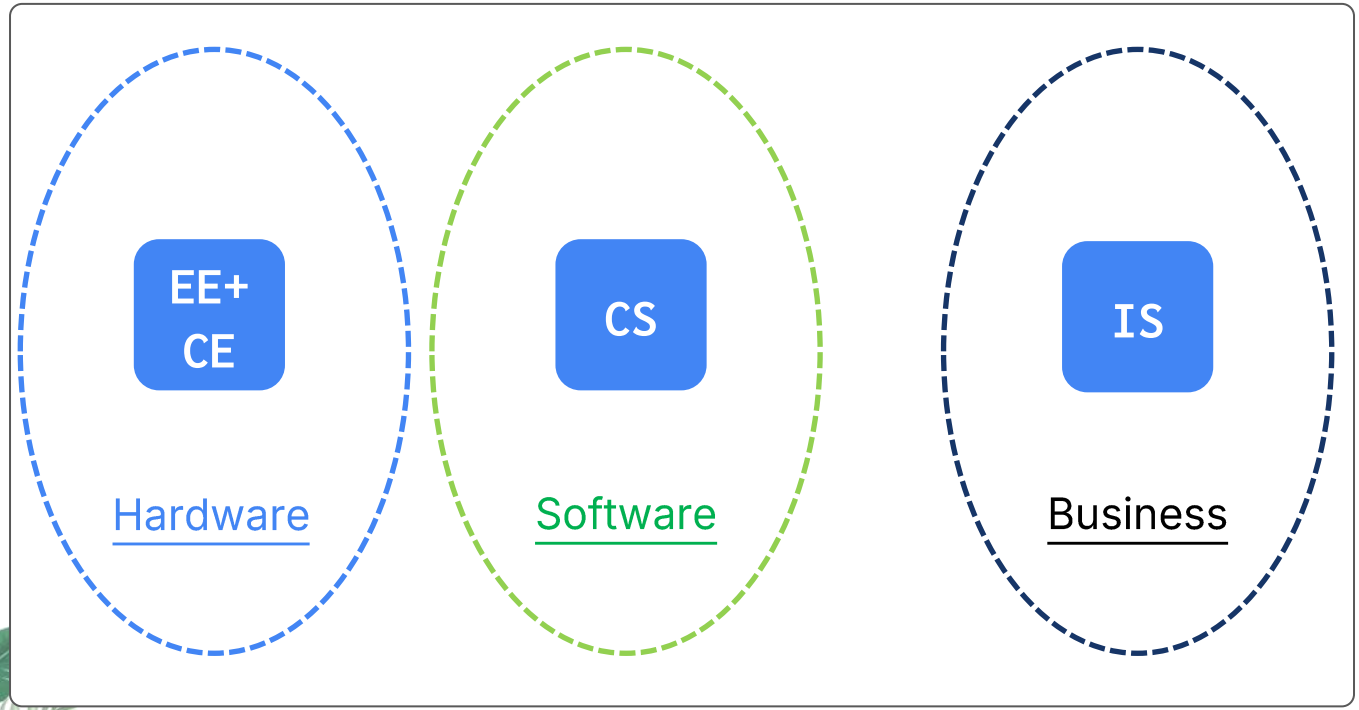
https://dora.dev/insights/balancing-ai-tensions/
https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/tech-forward/state-of-ai-trust-in-2026-shifting-to-the-agentic-era

Software Evolution

Course Overview

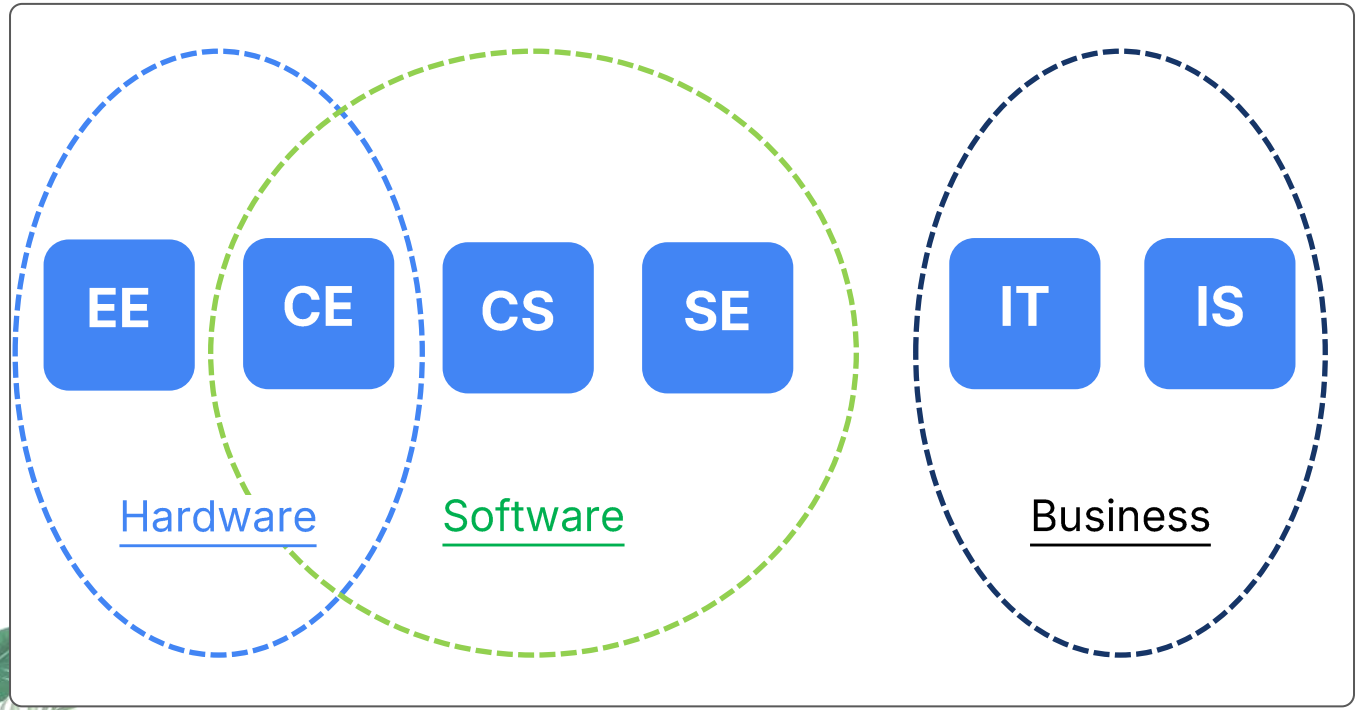
Computing Interrelationships

Post-1990s



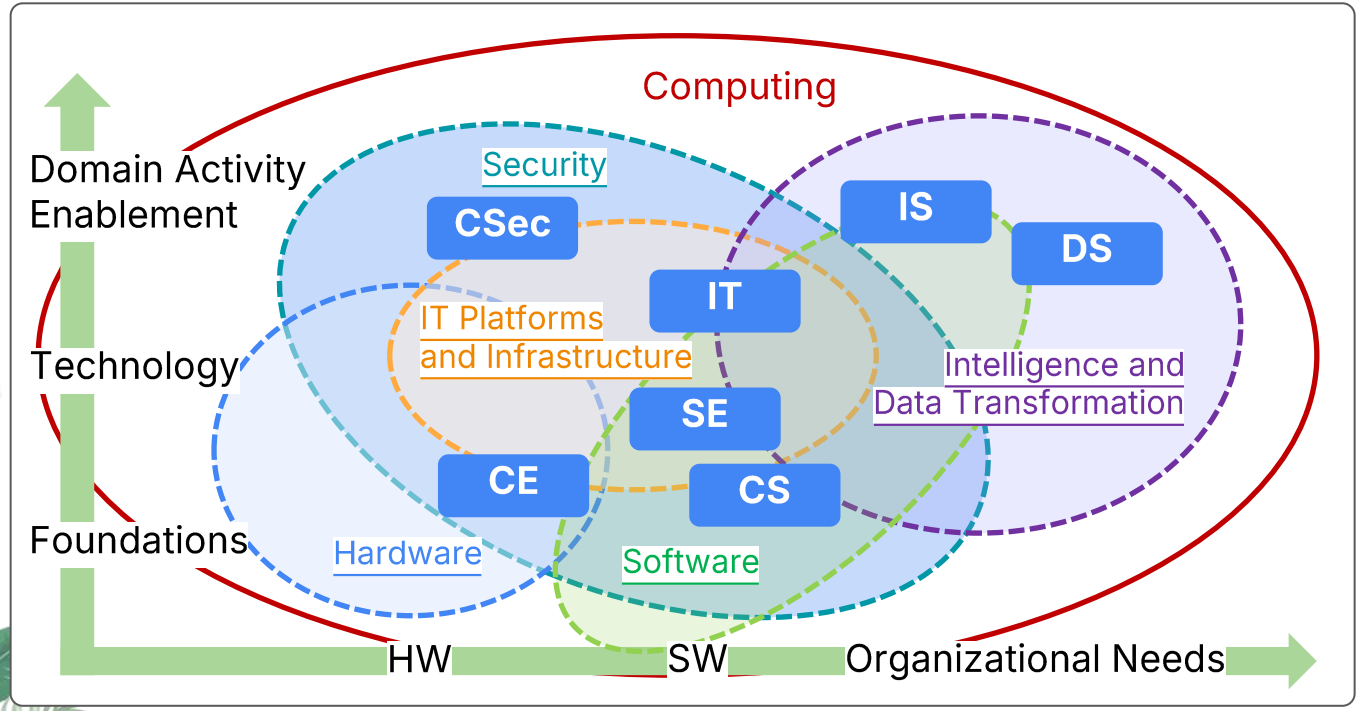
Computing Interrelationships

Pre-2010s



Computing Interrelationships

Post-2010s



AI?

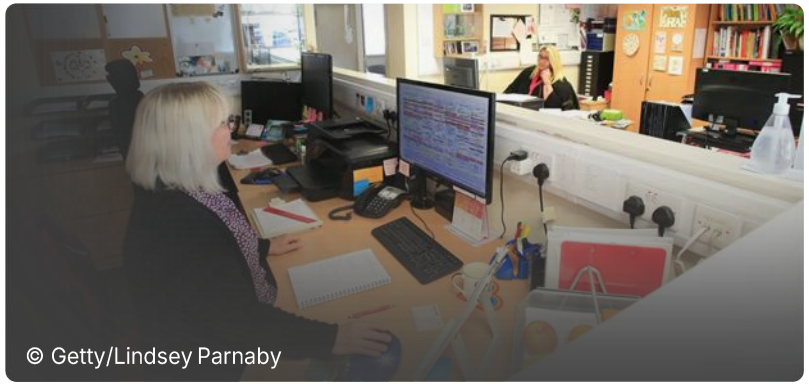
Course Overview



Computer Science

"academic discipline;
theories and methods"

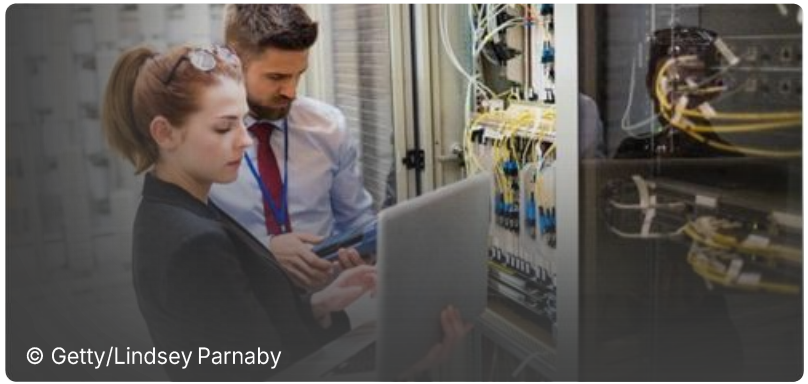
Course Overview



Information Systems

Organizational solutions from an organizational perspective.

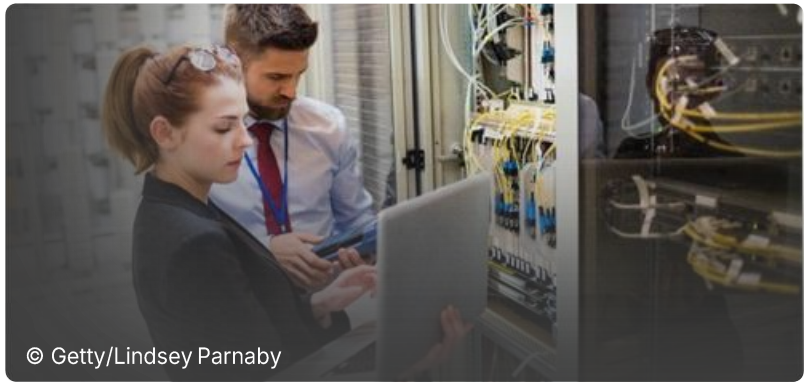
Course Overview



Information Technology

Organizational solutions from an infrastructural perspective.

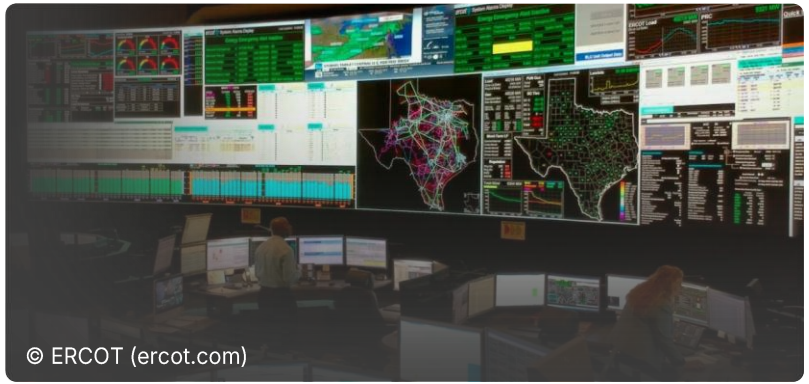
Course Overview



Information Technology

"...building and maintenance of ***computing Infrastructures***." (ACM and IEEE-CS, 2020)

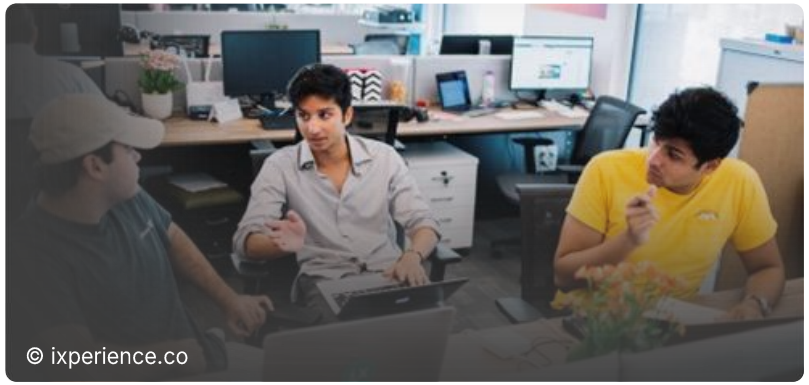
Course Overview



Systems Engineering

Complex CBS that involves HW/SW, policies, users, and its environment.

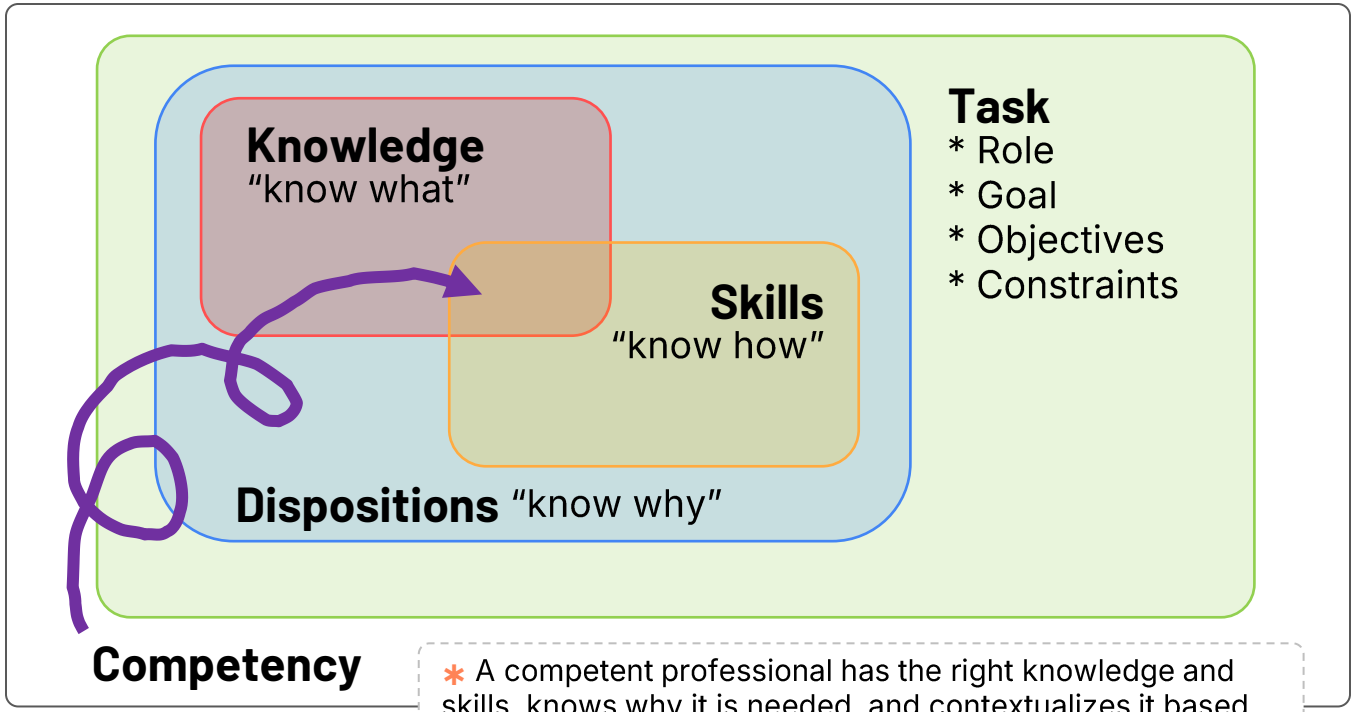
Course Overview



Software Engineering

Practical discipline; designing reliable, secure, and high-quality software.

Competency Model



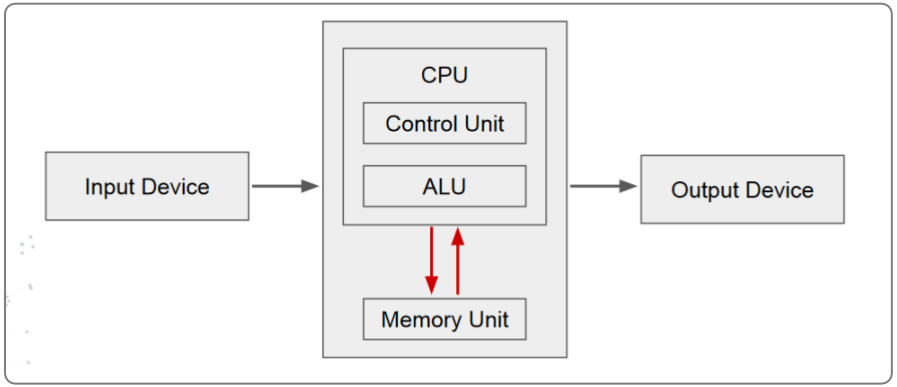
* A competent professional has the right knowledge and skills, knows why it is needed, and contextualizes it based on his assigned role, goals, objectives, and constraints.

Software Evolution

Software Evolution

What is a Computer?

It is a ***programmable*** **electronic machine** that accepts **digitized input**, processes according to a set of **internally stored instructions**, and produces the resulting **output information**.



* A block diagram of ***classical computing machine***, showing how I/O devices communicate with the CPU.

What is a Software?

- Programs are developed and used by the author (Jalote, 1997).
- Software or systems are used by other people.

What is a Software?

It is a ***set of programs*** that enables computers to perform a specific task.



* Let's try to open a "***software***" to see what is inside it...

What is a Software?



Open
Run as administrator
Open file location
Properties

AngryBuuurdz

> This PC > C: > Program Files > AngryBuuurdz



data EXE GDF DLL LUA DAT

* The software itself is not just a standalone executable, but rather a combination of files, configurations, and other artifacts.

What is a Software?

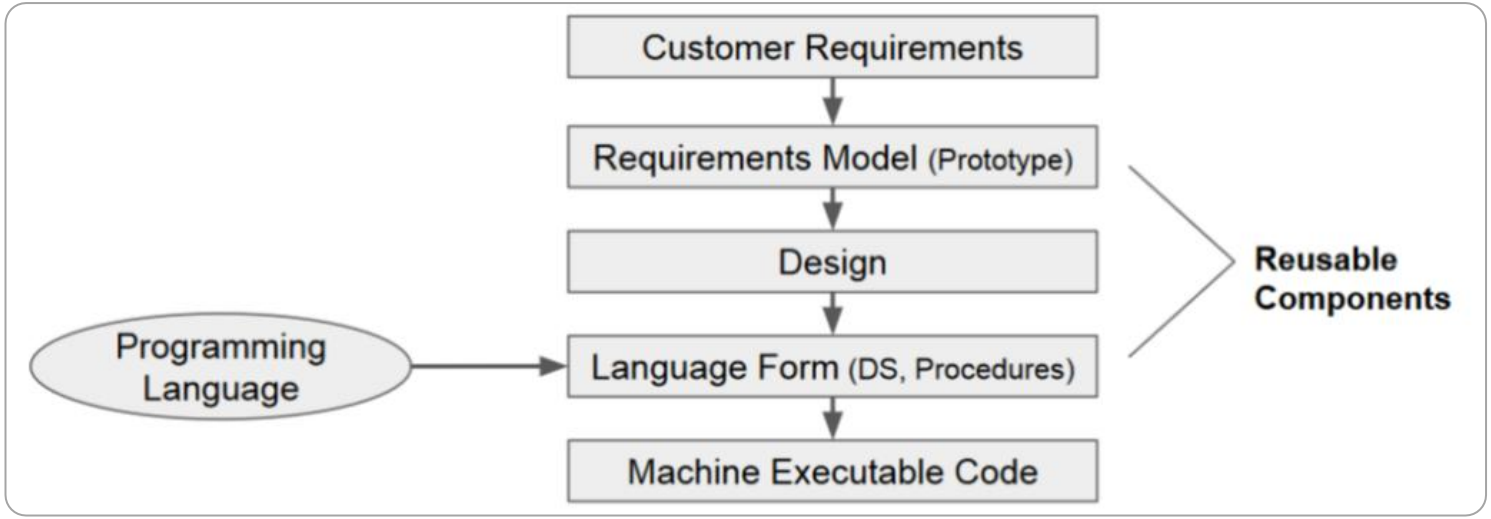
Includes the following:

- **set of programs**,
- **data** and **data structures**,
- **configuration files**, and
- **documentations** detailing its design, development, operation, and maintenance.

* However, only specific documentations such as user manuals are bundled for public use.

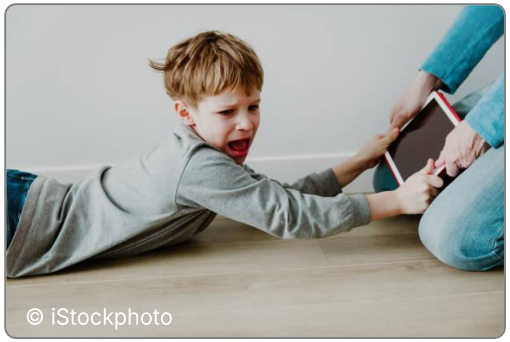


Software Components



* Analysis/Design models and source code are reusable.

History of Software

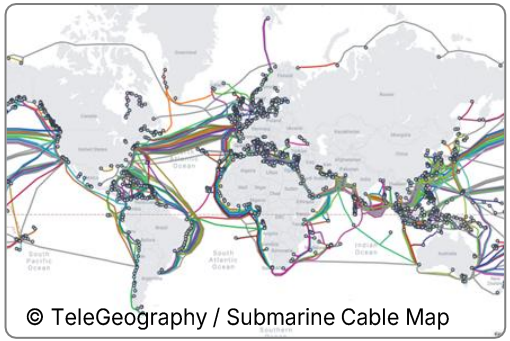












History of Software

1613

Human Computers

(Occupation)



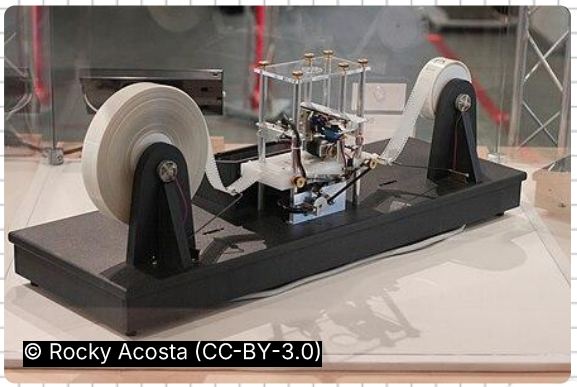
History of Software

Published “On Computable Numbers, with an Application to the Entscheidungsproblem (Decision Problem)” in 1937.

1936

Turing Machine

(Alan Turing)



Universal Computing Machine (Turing Machine)

A simple abstract computational device intended to help investigate the extent and limitations of what can be computed.

History of Software

Early Days

- Development efforts during WWII.
- Focused on scientific, military, and defense.
- Manage supply chain, ballistics, and cryptanalysis during the war.
- Independent research and progress.

History of Software

Early Days

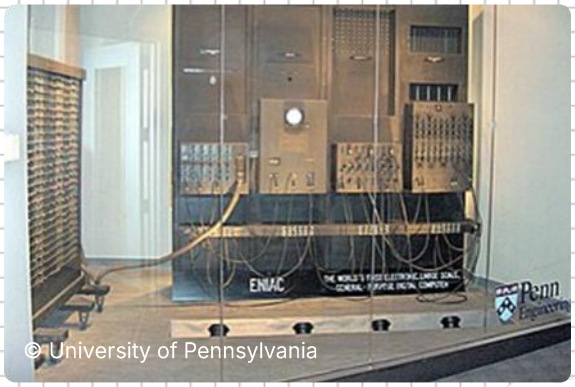
- Computers use punch cards and paper tapes as I/O.
- Perform batch operation and processing only.
- Software development required deep knowledge and understanding of ***machine language***.
- Software engineering is not yet a thing!

History of Software

1945

Plugboards

Electronic Numerical Integrator and Computer



von Neumann Architecture (Class

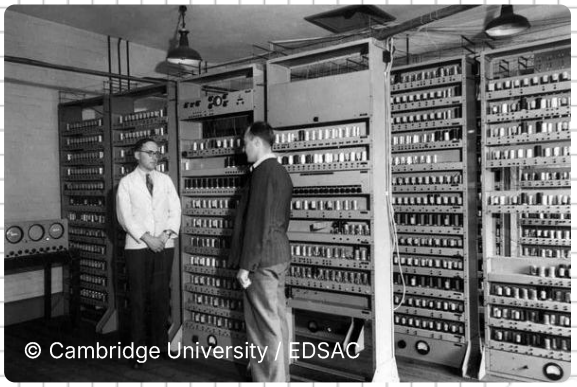
A computer architecture based on a 1945 description
by John von Neumann and others
in the First Draft of a Report on the EDVAC.

History of Software

1949

First Assembler

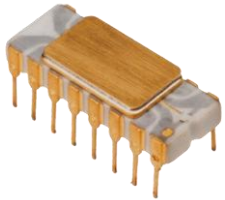
Electronic Delay Storage Automatic Computer



History of Software

The 1960s – Mainframe era

- Computers were expensive and bulky.
- Surge of OS and high-level programming languages.
- Lack of skilled engineers and the lack of competence and experience—severely understaffed position.
  - Software was built by ad hoc domain-knowledge experts w/o formal training.
  - Software aging and chronic disease.





History of Software

The 1970s – Microcomputer era

- Commercialization of early microprocessors.
- Small and affordable computers used in business and large institutions.
- Rise of more complex software and systems.



History of Software

The 1970s – Microcomputer era

- The formalization of "***software engineering***" started in 1968, 1972, and onwards.
- The materials for training ideal "***software engineers***."



History of Software

The 1980s – PC era

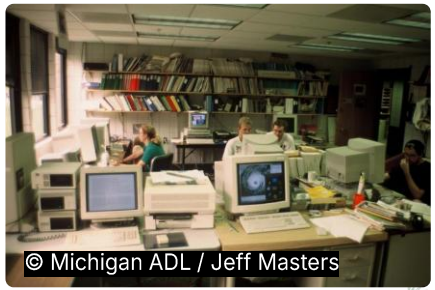
- ***Cheaper HW***: Widespread adoption of PC.
- Interaction through MS-DOS and early GUIs.
- Monolithic architectures and object-oriented approaches.
- Software with strong customer impact.
- Technology-driven businesses.



History of Software

The 1990s – Client-Server era

- Rise of internet and server-based systems.
- Shift to ***client-server models***: distributed monolithic architecture, growth of database systems, and multi-user access and real-time interactions.
- Business and technology integration.
- Popularity of email and web browsers.
- Growth of e-commerce.



History of Software

The 2000s – Web era

- ***Internet***: major platform for software delivery and end-user access.
- ***Internet-based systems***: rapid expansion of e-commerce and online services.
- Rise of cybersecurity vulnerabilities and threats.
- Counterfeit goods and piracy.



History of Software

The 2010s – Mobile era

- Rise of modern smartphones and tablets.
- On-the-go applications and mobile-first design.
- Dominance of Android and iOS operating systems.



History of Software

The 2020s – Cloud-Computing era

- Investments in cloud computing infrastructures.
- Cloud and serverless infrastructures:
  Everything-as-a-Service (XaaS) such Infrastructure-, Platform-, and Software-as-a-Service.
- Microservices, low-code, and AI-driven applications.



History of Software

The 2020s – Cloud-Computing era

- Strong focus on cybersecurity, privacy, and ethics.
- Niche-oriented and user-centered applications.
- "**Pay-as-you-go**" subscription models.
- Shared-source and custom licenses.



Software Evolution

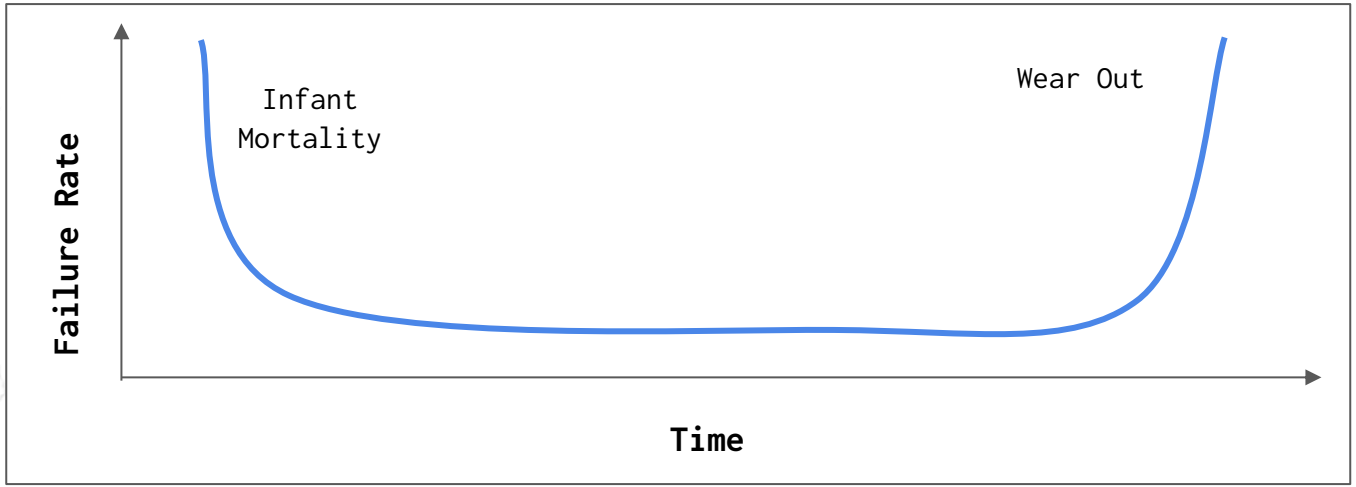
Software Characteristics

Software Characteristics

- Software...
  - is ***developed*** or ***engineered***,
  - is ***custom-built***, and
  - does not deteriorate.

Software Characteristics

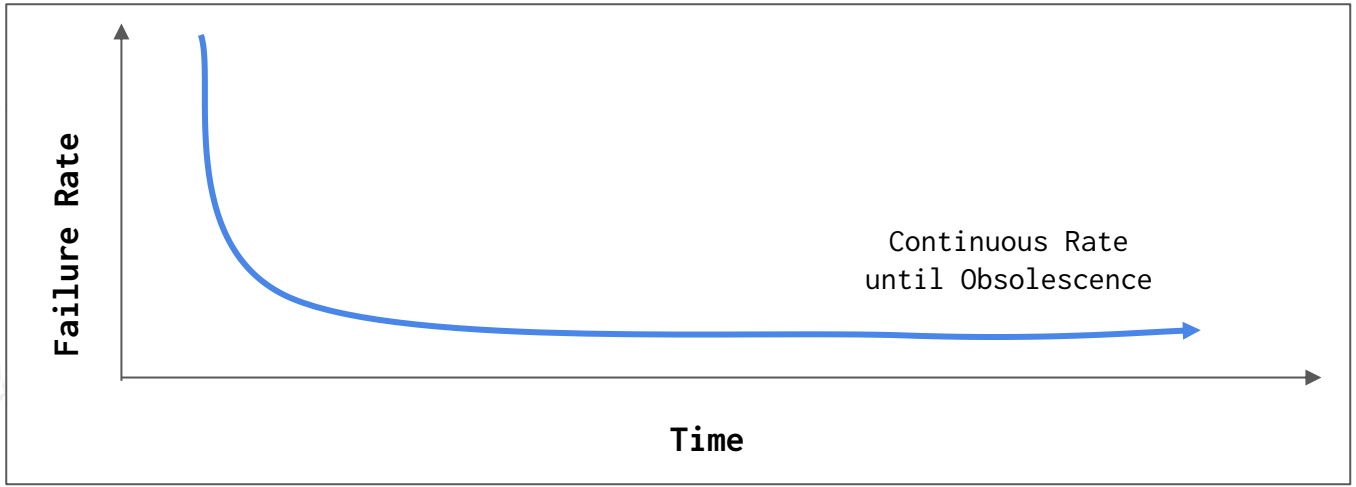
Hardware Failure Curve



* Hardware failure typically stabilize during its lifetime and wears out after its service life.

Software Characteristics

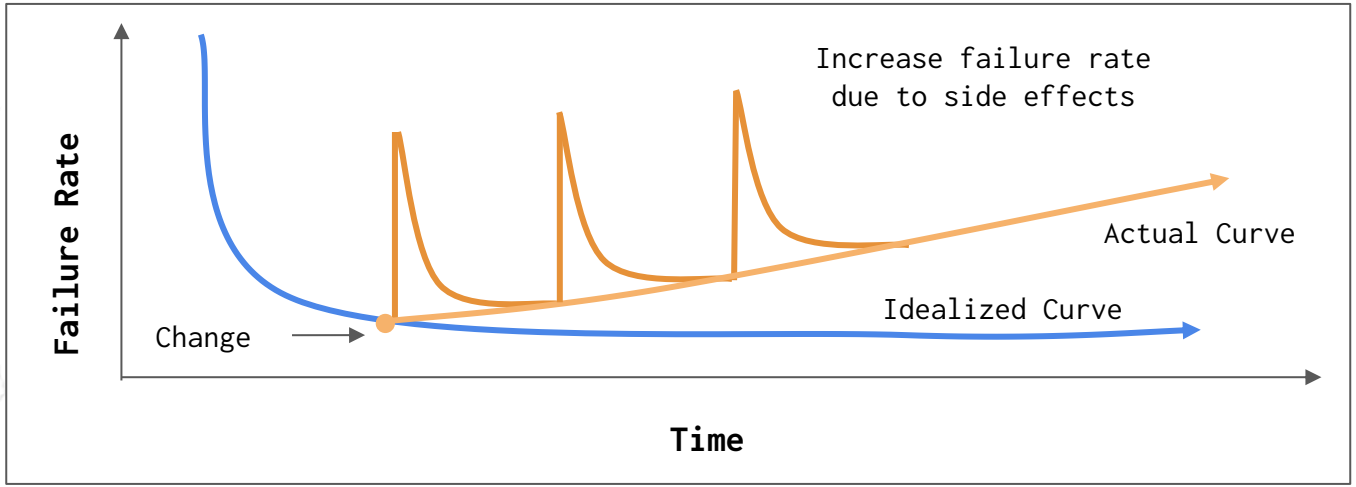
- Idealized Software Failure Curve



* Software does not physically wear off.

Software Characteristics

Actual Software Failure Curve



* However, software failure increases over time as changes introduce defects.

Software-Related Problems

#1

Hardware advances outpace software.

* Each year, a new hardware is released, software must catch up to ensure compatibility and reliability.



Software-Related Problems

#2
Demand outpaces development.

* The market needs drastically change based on multiple factors, old requirements may no longer meet new demands.



Software-Related Problems

#3

Continuous demand for quality and reliability.

* End-users are becoming more sophisticated and bad actors use more modern methods, software companies must strive to be a step ahead.



Software-Related Problems

#4
Inadequate resources and poor design.

* The economics of software development and implementation “readiness” are often questionable, as tech startups may bypass established SE practices.



Software Safety

- Typically, applicable to HW and embedded systems.
- Not increasing the risks of injury or fatality with the use of a device.
- Not a major focus of software engineering.

Software Quality

- Meeting or exceeding the customer expectations.
- One of the goals of software engineering.

Software Security

- Covers physical and cybersecurity.
- SW shouldn’t be used for criminal or malicious intents.
- Partial coverage in software testing.

Software Evolution

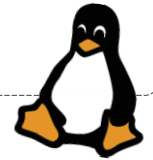
Software Classifications

Classification by Purpose

#1 System Software

The ***manager*** of a general-purpose computer system and enables the execution of other software.

- Includes BIOS/UEFI, bootloader, firmware, operating systems, and device drivers.
- Memory management, concurrency, parallel computing, HW and low-level interfaces.
- Electrical, Electronics & Communication, and Computer engineers.

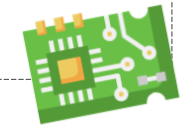


Classification by Purpose

#2 **Firmware Bundle**

A ***managed software bundle*** dedicated to a specialized HW and purpose, typically stored in a ROM or flash memory.

- Includes BIOS/UEFI, bootloader, firmware, operating systems, and device drivers.
- Memory management, concurrency, parallel computing, HW and low-level interfaces.
- Electrical, Electronics & Communication, and Computer engineers.



Classification by Purpose

#3 **Application Software (App)**

Covers a wide range of software used for specific tasks such as ***gaming***, ***entertainment***, or ***productivity***.

- For mobile, desktop, and web platforms.
- Knowledge in databases, high-level programming languages, and frameworks.
- Designers, IT, and Computer Science.





Classification by Purpose

#4 **Games**

An application designed for ***interactive play***, through a ***rule-based system*** to pursue defined ***goals***, ***challenges***, or ***outcomes***.

- For mobile, desktop, and web platforms.
- Game design/mechanics, interactive media/gfx, and software development skills.
- Designers, IT, and Computer Science.



Classification by Purpose

#5 **Enterprise (Business Software)**

A sub-type of applications designed to ***assist businesses;*** including management, HR, accounting, inventory, and sales.

- For mobile, desktop, and web platforms.
- Knowledge in databases, high-level programming languages, and frameworks.
- Designers, IT, and Computer Science.



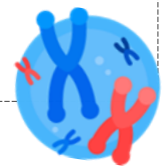


Classification by Purpose

#6 **Engineering and Scientific SW**

Specialized for complex ***calculations***, ***simulations***, and ***data analysis*** with high accuracy and precision.

- Desktop and web platforms.
- Domain-specific expertise and software development skills.
- STEM, IT, and Computer Science.



Classification by Purpose

#7 **Artificial Intelligence SW**

Complex and hybrid systems to mimic or even outperform human intelligence and efficiency for various tasks.

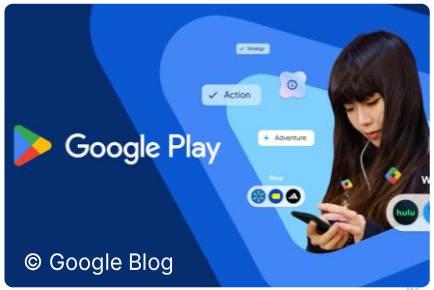
- Desktop and web platforms.
- Math knowledge, domain-specific expertise, and software development skills.
- STEM, IT, and Computer Science.



Target Market

#1 **Generic Software**

- Developed and sold by a software vendor.
- Diverse enterprise customers and direct end-users
  – niche-based apps on targeted market segments
  – typically sold in an application marketplace or vendor website
- Revenue based on adoption or in-app purchases.



Target Market

#2 **Customized Software**

- Commission by a client.
- Developed and maintained by a software contractor.
- Client-based specifications
  – stronger focus on quality, safety, and security.
  – software delivery is varied (on-premise/cloud-based)
- Revenue based on contracts



Sales Model

#1 **Software Product**

- Developed and sold by a software vendor.
- Traditional model:
  - buy rights to use software indefinitely (perpetual license)
  - typically, large upfront costs
  - one-time revenue
- Examples: Windows OS, Games, etc.



Sales Model

#2 **Software-as-a-Service**

- Developed and managed by a software provider.
- Software-as-a-Service model:
  - plans or subscription basis
  - not owned by the customer
  - sustainable revenue
- Examples: Microsoft 365, Netflix, etc.



© Google.com

Software Evolution

Roles in SW Development

Roles in Software Development



Stakeholders

All who impacts or are impacted by the software development.

Roles in Software Development



Project Team

Ad hoc team of experts.

Roles in Software Development



Client

Commissions a software.

Roles in Software Development



Customer

Buys software.

Roles in Software Development



End-user

Directly uses the software.

Summary

- Business as usual, but ***AI-driven***.
- The computing field is diversifying.
- Software continually evolves.
- Software is designed and developed.
- Software can be categorized based on purpose, target market, or sales model.
- Different roles in software development impacts or are impacted by the software.



Scan me!

Evaluation Form

forms.gle/eUbH7NiBsvQGaPt27

Thank You!

References

- Association for Computing Machinery. (2018). ACM Code of Ethics and Professional Conduct. https://www.acm.org/code-of-ethics
- Barnett, S. (2000). Pen Computing Magazine: Handspring Visor. http://www.pencomputing.com/palm/Reviews/visor1.html
- Bauer, F. L. (1972). Software Engineering. In Information Processing (p. 71).
- CC2020 Task Force. (2020). Computing Curricula 2020: Paradigms for Global Computing Education. Association for Computing Machinery.
- Duan, Y., Fu, G., Zhou, N., Sun, X., Narendra, N. C., & Hu, B. (2015). Everything as a Service (XaaS) on the Cloud: Origins, Current and Future Trends. 2015 IEEE 8th International Conference on Cloud Computing, 621–628. https://doi.org/10.1109/CLOUD.2015.88
- Gotterbarn, D., Miller, K., & Rogerson, S. (1997). Software engineering code of ethics. Communications of the ACM, 40(11), 110–118. https://doi.org/10.1145/265684.265699
- Hwang, K.-T., & Kim, N.-Y. (2004). System for Anti-Piracy of Software under Windows Operating System. The KIPS Transactions:PartD, 11D(2), 423–434. https://doi.org/10.3745/KIPSTD.2004.11D.2.423
- IBPAP. (2022). THE BIG PH IT-BPM LEAP The Philippine IT-BPM Industry Roadmap 2028 Executive Summary.

References

- IEEE. (1990). IEEE Standard Glossary of Software Engineering Terminology. IEEE Std 610.12-1990, 1–84. IEEE Std 610.12-1990. https://doi.org/10.1109/IEEESTD.1990.101064
- IEEE Computer Society. (2014). Guide to the Software Engineering Body of Knowledge (SWEBOK®): Version 3.0 (P. Bourque & R. E. Fairley, Eds.). IEEE Computer Society Press.
- Jones, C. (2013). The Technical and Social History of Software Engineering. Addison-Wesley.
- Larman, C., & Basili, V. R. (2003). Iterative and incremental developments. A brief history. Computer, 36(6), 47–56. Computer. https://doi.org/10.1109/MC.2003.1204375
- Manilla Bulletin. (2000). Microsoft cracks down on Filipino illegal software. ZDNET. https://www.zdnet.com/article/microsoft-cracks-down-on-filipino-illegal-software/
- NEDA. (2022). PHILIPPINE DEVELOPMENT PLAN 2023-2028. National Economic and Development Authority.
- PSIA. (2019). Philippine IT and Software Environment.
- Shafiei, H., Khonsari, A., & Mousavi, P. (2022). Serverless Computing: A Survey of Opportunities, Challenges, and Applications. ACM Computing Surveys, 54(11s), 239:1-239:32. https://doi.org/10.1145/3510611
- Thomasson, M. (2019). Video Game Cartridges: The History of Durable, Removable, and Portable Software. In The Routledge Companion to Media Technology and Obsolescence. Routledge.
- Turing, A. M. (1937). On Computable Numbers, with an Application to the Entscheidungsproblem. Proceedings of the London Mathematical Society, s2-42(1), 230–265. https://doi.org/10.1112/plms/s2-42.1.230